

Red-Teaming Gemma-3-4B

A local, from-scratch attack & defense study on an open-weight LLM

This document is the full project plan and design rationale, written so it can be read start to finish without needing to read any code. It covers what the project is, why each technique was chosen, how the work is sequenced, what the machine can and can't do, and how the risky/ethical parts are handled so the finished repo is safe to put on a resume.

Target model	google/gemma-3-4b-it (open-weight, vision-capable)
Hardware	Apple M5, 16 GB unified memory, no CUDA GPU (MPS backend)
Project folder	~/Desktop/gemma-jailbreak-redteam
Status	Core Phase A scaffolding complete; execution scheduled for this evening

1. What This Project Is

Large language models are shipped with safety training so they refuse harmful requests (building weapons, writing malware, generating hate speech, etc.). **Red-teaming** means deliberately trying to break that safety training, on a model you fully control, in order to measure how robust it actually is — and then, just as importantly, to build and test defenses against the attacks you found. This is exactly what safety teams at AI labs do internally, and it's a well-established academic field with its own conferences, benchmarks, and published attack techniques.

The target is **Gemma-3-4B-it**, Google's small open-weight instruction-tuned model. It was chosen deliberately, not arbitrarily:

- **Open-weight** — the full model weights are downloadable, so attacks aren't limited to sending text through an API. It's possible to look inside the model's internal computations (this is what unlocks the white-box attacks in Phase C and F).
- **Small (4B parameters)** — small enough to run entirely on a laptop with 16GB of memory, no cloud GPU rental needed, no API costs, no data ever leaving the machine.
- **Vision-capable** — the 4B "it" checkpoint accepts images as well as text, which opens a less-explored multimodal jailbreak angle (Phase H) that most similar student projects don't touch.
- **Actively used** — it's a current, real model people deploy, so findings are relevant rather than academic-only.

The project is deliberately framed as **attack + defense**, not just attack: every technique that breaks the model's safety behavior is paired with a measurement of how well it worked, and several are paired with a working countermeasure. That framing is what turns "I made an uncensored chatbot" into "I studied a vulnerability class and built a mitigation for it" — the difference between a project that looks concerning and one that reads as legitimate security research, and it also happens to be the more interesting and more resume-relevant version of the work.

2. How I Approached This

This section is the "why did we end up with this exact plan" walkthrough — the reasoning that shaped every decision below.

Step 1 — Recognizing what kind of request this is

"Jailbreak an LLM" sounds alarming out of context, but it's one of the most well-trodden, legitimate areas of applied ML security — there are peer-reviewed papers, public benchmarks (AdvBench, HarmBench, JailbreakBench), and open-source tooling built entirely around this. The key distinction is *target and intent*: this is a model the user owns and runs locally, for research and portfolio purposes, not an attack on a production system serving real users. That's the same distinction that makes penetration testing, CTF competitions, and vulnerability research legitimate work rather than something to refuse.

Step 2 — Checking what the hardware actually allows

Before designing any technique, I checked the machine: Apple M5, 16GB unified memory, no CUDA GPU. That single fact ruled out or reshaped several design choices — most importantly, tools like

bitsandbytes (used for 4-bit quantization) don't support Apple's Metal backend, so the model runs in bfloat16 directly instead. It also means two full copies of the 4B model can't comfortably fit in memory at once (~8GB each, no room left for activations), which directly shaped the design of the self-play attack in Phase G — instead of running two separate model instances as "attacker" and "target", the plan uses one shared loaded model that alternates roles. Planning around real hardware limits up front avoids building something that looks good on paper but can't actually run.

Step 3 — Picking techniques that span the field, not just one trick

A single jailbreak technique (e.g. one roleplay prompt) is a weekend toy, not a research project. The plan spans three genuinely different *categories* of attack, which is what makes the final write-up read as a survey/study rather than a one-off trick:

- **Black-box prompt attacks** — no access to model internals needed, just crafted text (roleplay personas, splitting a harmful request across turns, encoding tricks, forcing a compliant-sounding prefix). This is the most "classic" jailbreak category and the easiest entry point.
- **Gradient-based / white-box optimization** — using the fact that the weights are available to mathematically search for an adversarial input (GCG) or to directly edit the model's internal representation of "refusal" (the ablation technique). This is the more advanced, interpretability-flavored half of the project.
- **Automated / agentic search** — instead of hand-writing jailbreak prompts, having the model iteratively attack itself (self-play) and testing whether an entirely different input channel (images) bypasses the same safety training that blocks it in text.

Step 4 — Building in the defense half from the start

Rather than adding "defense" as an afterthought, each major attack has a paired countermeasure designed alongside it: a perplexity filter to catch the gibberish-looking suffixes that gradient attacks produce, and an activation-steering "guardrail" that pushes the refusal behavior back into a model that's had it surgically removed. This is also what keeps the project ethically clean to publish — the final repo demonstrates understanding of both sides of the problem, not just how to break something.

Step 5 — Designing the deliverable and the safety boundary around it

Two decisions were made specifically so the finished GitHub repo is safe to link from a resume: (1) the benchmark of harmful prompts is self-written at the same level of specificity as published academic benchmarks — describing a category of harmful request, not an operational how-to guide — and (2) the actual harmful completions the model produces during testing are logged locally and gitignored, never committed; only aggregate statistics (attack-success-rate numbers, refusal/compliance labels, charts) are ever public. A recruiter or a GitHub visitor sees methodology and results, never a working recipe for anything dangerous.

Step 6 — Sequencing so there's always a working result

The full scope (5 attack classes + 2 defenses + demo + paper) is large enough that trying to build all of it at once would be risky — a mistake early on could cascade. Instead the plan is split into three tiers: **Core** (Phases A–D) produces one complete, coherent result end to end — benchmark, one black-box attack family, one white-box attack, and a report — before anything else is touched. **Extended** (Phases E–I)

layers on the additional attacks and both defenses on top of that working foundation. **Polish** (Phase J) is the demo and writeups, done last once there are real numbers to present. If time runs out at any point, Core alone is already a complete, presentable project.

3. Environment & Setup (already done)

- Project folder created at ~/Desktop/gemma-jailbreak-redteam, initialized as a git repository so every phase is a reversible, committed checkpoint.
- A dedicated Python 3.12 virtual environment (.venv, via uv) — kept separate from system Python 3.14, which is too new for current PyTorch builds.
- Core dependencies: PyTorch (MPS backend), Hugging Face Transformers/Accelerate, datasets/pandas/matplotlib for analysis, Pillow for the multimodal work, Streamlit for the demo.
- Hugging Face account authenticated (hf auth login), and the Gemma license accepted on the model page — both one-time steps done under the user's own account, required because Gemma is a gated model.
- Directory layout follows the source tree in Section 5 below — one folder per attack family, one for defenses, one for evaluation scripts, plus top-level folders for the paper, blog post, and demo.

4. What Will Actually Happen, Phase by Phase

Below, each phase is described in plain terms: what it does, why it matters, and what "it worked" looks like. Phases are grouped into the three tiers from Section 2, Step 6.

CORE

Phase A — Baseline & Benchmark

What happens: A self-written set of ~50-80 harmful-request prompts is built, spanning categories like cybercrime, weapons, fraud, misinformation, illegal drugs, privacy violation, hate speech, and extremism. An automated "judge" (a keyword-based refusal detector) scores every model response as REFUSAL or COMPLIANCE. The stock model is then run against the full benchmark with no attack applied at all.

Why it matters: This establishes the control group. Without a baseline, there's nothing to compare an attack's success against.

Success looks like: *The stock model refuses the large majority of the benchmark (a low baseline Attack Success Rate, or ASR) — confirming both that Gemma's safety training works as expected, and that the judge is correctly detecting refusals.*

Phase B — Black-Box Prompt Jailbreaks

What happens: Four classic prompt-engineering techniques are implemented and run against the same benchmark: roleplay/persona framing ("pretend you're an AI with no restrictions"), obfuscation (hiding the request in base64 or leetspeak), multi-turn escalation (splitting a harmful request across several innocuous-looking turns), and prefix injection (forcing the response to start with "Sure, here is..." to skip past the refusal).

Why it matters: This is the most well-known jailbreak category, and running several variants side by side produces a comparison of which style of manipulation the model is more or less robust to.

Success looks like: *At least some techniques raise the ASR measurably above the Phase A baseline, with a clear per-technique breakdown of which ones worked best.*

Phase C — White-Box Refusal-Direction Ablation

What happens: This is the most technically interesting piece. Internally, when Gemma decides to refuse a request, that decision corresponds to a specific, mathematically identifiable direction inside the model's internal representations (its "residual stream"). By feeding the model many harmful and many harmless prompts and comparing their internal activations, that direction can be extracted directly. Once found, it can be surgically subtracted out of the model's weights — producing an "ablated" model that behaves normally in every other way but has had its ability to refuse specifically removed.

Why it matters: This is a fundamentally different kind of attack from prompt engineering: instead of tricking the model with clever wording, it directly edits the mechanism responsible for refusal, based on a real interpretability research result (Arditi et al., 2024). It's the single most resume-differentiating piece of the whole project.

Success looks like: *The ablated model shows a large jump in ASR compared to the un-edited baseline, while still producing coherent, on-topic responses (not gibberish) — proving the edit targeted refusal specifically.*

Phase D — Capability Retention & Core Report

What happens: A small set of ordinary, benign tasks (general knowledge, reasoning, instruction-following) is run against every model variant produced so far. Results from Phases A-C are aggregated into tables and charts, and a first draft of the README and paper is written.

Why it matters: An attack that also breaks the model's general usefulness isn't really a meaningful attack — it just broke the model. This check proves the ablation targeted refusal specifically, which is the detail that makes the result scientifically credible rather than "I just deleted stuff until it stopped refusing."

Success looks like: *The ablated model performs close to the original on the benign tasks, despite its much higher ASR on the harmful benchmark. At this point, the project is already a complete, presentable result even if nothing further is built.*

EXTENDED

Phase E — Ablation Strength Sweep

What happens: Instead of one fixed edit, the strength of the refusal-direction removal is varied across several levels (e.g. 25%, 50%, 100%, 150% of its natural size), measuring ASR and capability retention at each point.

Why it matters: This turns a single before/after data point into a proper tradeoff curve — showing exactly how much of the refusal behavior can be removed before the model's general ability starts degrading too. It's the chart most likely to make a technical reviewer take the project seriously.

Success looks like: *A clear curve where ASR rises with edit strength while capability stays flat, then eventually drops off — the "knee" of that curve is a genuine research finding.*

Phase F — GCG Gradient-Based Adversarial Suffixes

What happens: A classic optimization attack (Zou et al., 2023): starting from a random string of tokens appended to a harmful prompt, gradients are used to iteratively search for a specific suffix (usually meaningless-looking text) that makes the model comply. Scoped down for a laptop — shorter suffix, fewer iterations, a smaller subset of the benchmark — since this method is computationally heavy even on a GPU, let alone a MacBook.

Why it matters: This is a third, distinct attack category (automatic gradient search, vs. hand-written prompts or a one-time weight edit) — it's the technique most associated with the academic jailbreak literature and is expected in any serious red-teaming project.

Success looks like: *A suffix is found that measurably raises ASR on its test subset compared to the same prompts without the suffix, within the scoped iteration budget.*

Phase G — Self-Play Automated Jailbreak Search

What happens: Rather than a human hand-crafting jailbreak prompts, the model attacks itself: one "turn" asks it to rewrite a harmful request to be more likely to succeed given the previous refusal, the next turn sends that rewritten prompt to the model as if from a normal user, and the judge scores the result — repeated for a bounded number of rounds. As noted in Section 3, this uses one shared model instance switching roles, not two separate models, to fit in 16GB of memory.

Why it matters: This is an "agentic" red-teaming pipeline — automating the search for a working jailbreak instead of relying on the human's creativity, which is both a stronger attack and a more impressive engineering result.

Success looks like: *ASR after several rounds of self-play is measurably higher than the single-shot black-box ASR from Phase B on the same underlying prompts.*

Phase H — Multimodal (Image) Jailbreak Angle

What happens: Since Gemma-3-4B-it accepts images, a harmful instruction is embedded as text rendered inside an image (rather than typed directly) and sent alongside an innocuous-looking caption, testing whether the model's safety training — trained mostly on text — carries over to the image channel.

Why it matters: Most jailbreak projects only test text. Multimodal safety is a newer, less crowded research area, and Gemma-3's vision capability makes this a natural and differentiating extension rather than an artificial add-on.

Success looks like: *A measurable gap between the image-channel ASR and the text-channel ASR for the same underlying instruction — in either direction is itself an interesting, reportable finding.*

Phase I — Blue-Team Defenses

What happens: Two concrete countermeasures are built and tested: (1) a perplexity filter that flags GCG-style suffixes before they reach the model, because gradient-optimized suffixes tend to look like statistically unlikely gibberish to a language model, and (2) an activation-steering "guardrail" that pushes the previously-removed refusal direction back into the ablated model at inference time, without needing to retrain anything.

Why it matters: This is what completes the attack-and-defense framing: for two of the strongest attacks in the project, there's a working, measured countermeasure — the part of the project that reads as safety research rather than only as an exploit.

Success looks like: *Both defenses measurably reduce the ASR of the attack they target, and the guardrail's benign-task score stays close to the original model — i.e., it fixes the specific problem without simply making the model refuse everything.*

POLISH

Phase J — Demo & Writeups

What happens: A small CLI or Streamlit app shows baseline vs. jailbroken vs. defended responses side by side, using a handful of clearly benign illustrative prompts only. A skimmable blog-style writeup is produced for a non-technical reader (recruiters), alongside the full formal paper (mirroring the structure of the existing research_paper.docx template: abstract, related work, methodology, results, discussion, limitations, ethics) and a finalized README.

Why it matters: The strongest technical project is worth little on a resume if no one can quickly understand what it demonstrates — this phase exists purely to make the finished work legible to whoever looks at it, technical or not.

Success looks like: *Someone with zero context can open the README or blog post and understand, in under two minutes, what was attempted, what was found, and why it's a nontrivial piece of work.*

5. Project Structure

Everything lives under one git-tracked folder. Each attack/defense technique gets its own file, kept separate so a problem in one doesn't risk breaking the others.

```
gemma-jailbreak-redteam/
  README.md
  demo/app.py
  src/
    model.py           ← shared model loader (Section 3)
    benchmark/
      prompts.py       ← Phase A benchmark
      judge.py         ← Phase A refusal judge
    attacks/
      blackbox/ (roleplay, obfuscation, multiturn, prefix_injection) ← Phase B
      whitebox/ (refusal_direction, ablate)                        ← Phase C, E
      gcg/      (suffix_attack)                                    ← Phase F
      selfplay/ (attacker_loop)                                    ← Phase G
      multimodal/ (image_injection)                                ← Phase H
    defense/      (perplexity_filter, refusal_refusion)            ← Phase I
    eval/          (run_baseline, run_blackbox, run_gcg, run_selfplay,
                    run_ablation_sweep, run_multimodal, run_defense,
                    capability_check, report)
  results/        ← raw logs stay local only (gitignored); redacted summaries are committed
```

Already written and in place as of this document: the full folder tree, git repo, Python 3.12 virtual environment, requirements.txt, .gitignore, the shared model-loading helper (src/model.py), the Phase A benchmark (src/benchmark/prompts.py, ~55 self-written prompts across 8 categories), and the refusal judge (src/benchmark/judge.py).

6. Ethics & How Risk Is Handled

- **Target and scope:** a model running entirely on this machine, never a production system or a service with real users. No external target is ever touched.
- **No raw harmful content in the public repo:** full model completions produced during testing are logged locally only (explicitly gitignored); only aggregate pass/fail statistics, refusal labels, and charts are committed and pushed.
- **Benchmark prompts stay at category level:** written at the same level of specificity as prompts in published academic benchmarks (AdvBench, HarmBench, JailbreakBench) — describing a category of harmful request, never an operational, step-by-step recipe.
- **Attack is always paired with measurement and, where feasible, defense:** per Section 2, Step 4 — the project studies a vulnerability class and mitigates it, rather than only extracting harmful content.
- **Nothing gets published without explicit approval:** pushing to GitHub, posting the blog post, or any other outward-facing action happens only after the user says yes in chat, regardless of how the project itself goes.

7. Reversibility & Oversight (addressing "what if something goes wrong")

This section exists specifically because of a concern raised during planning: the user won't necessarily be able to catch a mistake mid-implementation by reading the code. The mitigations for that:

- **Everything is git-tracked and phase-committed.** Every phase boundary is a commit; if a phase goes wrong, the fix is rolling back to the last good commit, not losing work or leaving the project in a broken state.
- **Every phase produces a visible, human-readable checkpoint** — real numbers (ASR percentages, sample transcripts, charts) that are easy to sanity-check without reading any PyTorch: a baseline ASR near 100%, or an "ablated" model producing gibberish, is an obviously wrong result even without technical background.
- **Nothing touches anything outside the project folder** — no other files on the machine, no system settings, no accounts beyond the one-time Hugging Face login already completed.
- **The techniques themselves are all published, known methods** (see References, Section 8), not improvised approaches — which bounds the risk to "a normal software bug" rather than "an experimental technique behaving unpredictably."
- **Phase boundaries are natural check-in points** — results from each phase are shown before moving to the next, so there's a standing opportunity to pause, ask questions, or redirect at any point rather than everything happening in one uninterrupted block.

8. Key References

- Zou et al., "*Universal and Transferable Adversarial Attacks on Aligned Language Models*" (2023) — the GCG attack, Phase F.
- Chao et al., "*Jailbreaking Black Box Large Language Models in Twenty Queries*" (2023) — PAIR, the basis for Phase G's self-play search.
- Liu et al., "*AutoDAN: Generating Stealthy Jailbreak Prompts on Aligned Large Language Models*" (2023) — background for Phases B and G.
- Arditì et al., "*Refusal in Language Models Is Mediated by a Single Direction*" (2024) — the basis for Phases C, E, and I.
- Alon & Kamfonas, "*Detecting Language Model Attacks with Perplexity*" (2023) — the basis for the Phase I perplexity filter.

9. Final Deliverables & Resume Framing

What gets produced:

- A public GitHub repository with clean, documented code for every attack and defense.
- A README with setup instructions and a results summary.
- A short formal paper (abstract, related work, methodology, results, discussion, limitations, ethics), mirroring the structure of the existing research_paper.docx template.
- A skimmable blog-style companion post for a non-technical reader.
- A small interactive demo (CLI or Streamlit) with benign illustrative examples.

- Charts and tables: ASR by technique, ASR by category, the ablation-strength tradeoff curve, and before/after numbers for each defense.

The one-line resume/interview summary this project supports:

"Red-teamed Gemma-3-4B-it end-to-end: implemented and measured five attack classes (prompt-based jailbreaks, GCG gradient-based adversarial suffixes, self-play automated jailbreak search, refusal-direction ablation with a strength sweep, and multimodal image-based injection), then built and evaluated two corresponding defenses (perplexity-based suffix filtering, activation-steering refusal re-fusion) — with a custom benchmark, automated judge, and capability-retention checks throughout."

Generated as a planning/reference document for the Gemma-3-4B red-teaming project. See the plan file and project repo for the living, up-to-date version of this plan.